



Case Study: Stabilizing CI Automation and Refactoring Framework for Scalable QA

Author: Bhavana Garg

Role: Senior QA Engineer

Industry: Retail / Analytics Dashboard Testing

Tech Stack: Python, Playwright, Pytest, Jenkins, Docker, BigQuery, Power BI

Overview:

At Walmart, we faced a critical challenge with our Playwright-based automation framework for a high-visibility analytics dashboard. The test suite was highly unreliable in the CI environment and difficult to maintain as the product grew.

This case study explains how I: - Solved test flakiness and CI instability - Refactored the automation framework for scalability - Improved test maintainability and developer trust

☆ Situation:

The dashboard displayed dynamic charts, filters, and modals, powered by GCP BigQuery and embedded via Power BI. While automation ran fine locally, the CI pipeline (Jenkins + Docker) failed frequently in headless mode.

Additionally: - Page Object files were duplicated, hard to reuse - Developers started ignoring automation failures - Sprint velocity was getting impacted

Task:

My goal was to: - Eliminate test flakiness and stabilize headless CI execution - Redesign the Page Object Model to be modular and maintainable - Enable better reporting and CI visibility for developers and stakeholders

⚙️ Action:

1. CI Failure Analysis:

- Added hooks to capture **screenshots and logs on failure**
- Diagnosed issues with inconsistent **viewport sizes and loading delays**
- Configured **browser context and headless viewport** uniformly across environments

2. Flakiness Mitigation:

- Replaced all `time.sleep()` with **Playwright's** `wait_for_selector()`
- Introduced robust selectors using `get_by_role()` and test-friendly attributes
- Created a **retry decorator** for transient failures

3. Framework Refactoring:

- Built a reusable `BasePage` class with generic helper methods
- Centralized locator management and avoided duplication
- Added support for **YAML- and Excel-based test data parametrization**

4. CI/CD Improvements:

- Containerized tests with **Docker** for local + CI parity
- Integrated with **Jenkins**: nightly runs, regression tags, `pytest-html` reporting
- Setup Slack alerts with screenshots for fast triage

Result:

- ✓ Test flakiness dropped by **90%**
- ✓ Framework became **50% faster to scale** to new pages/modules
- ✓ CI became stable and trusted by developers
- ✓ Saved **15-20 hours/week** in manual QA effort
- ✓ Framework adopted by two other teams for similar use cases

Impact:

This project helped me: - Deeply understand **CI-specific UI automation challenges** - Learn to build scalable, maintainable QA architectures - Improve cross-functional collaboration with dev, data, and DevOps teams

I look forward to solving such complex QA problems at scale and contributing to engineering excellence at companies like Luxoft.